

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 8: Inheritance

Course Taught by: Zubair Irshad
Courtesy of some slides: Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)



Logistics

1. Midterm Grades – plan to get done and will post by early next week and will be open for discussion next week's office hours!
 1. Overall, the performance looks good so far for most of you
 2. One general feedback: I think it might be good to learn the art of debugging.
 1. Reproducing error, analyze error messages, logs, stack traces
 2. Advanced: Break points in code, Stepping into code, variable inspection etc.
2. Class and Lab Quiz 3 dates
 1. Planning either next Wednesday or the follow-up Monday for Class Quiz 3
 2. Planning next to next Thursday for Lab Quiz 3
 3. Topics: Classes and Objects, Operator Overloading, Rule of 3, Linked List



What have we covered so far!

1. First quarter i.e. Basics
 1. Introduction to C++, Loops, Arrays, Functions
2. Second part i.e. Tools needed for C++ and OOP
 1. Pointers (1D and 2D)
3. Third part i.e. Core of C++
 1. Classes and Objects, Operator Overloading, Rule of 3
 2. Example of Classes and tools in action: Linked list
4. For post mid-term, the core is important so make sure you have understood it well!



Unit Outline

1. Introduction
2. Inheritance
3. Inheritance Hierarchy
4. Question Class
5. Derived Classes
6. Exercises



Inheritance

- **Inheritance** is a key concept in object-oriented programming that defines a relationship between a **general class** (called the *base class*) and a **specialized class** (called the *derived class*).
- The derived class **inherits** both **data** and **behavior (functions)** from the base class.
- This allows **code reuse**, **extension**, and **specialization** of existing classes.
- Common terminology:
 - *Base class* ↔ *Derived class*
 - *Superclass* ↔ *Subclass*
 - *Parent class* ↔ *Child class*
- Inheritance represents an **“is-a” relationship** — meaning the derived class is a more specific type of the base class.
 - The **base class** defines general features or behavior.
 - The **derived class** extends or customizes those features.



Motivation for Inheritance

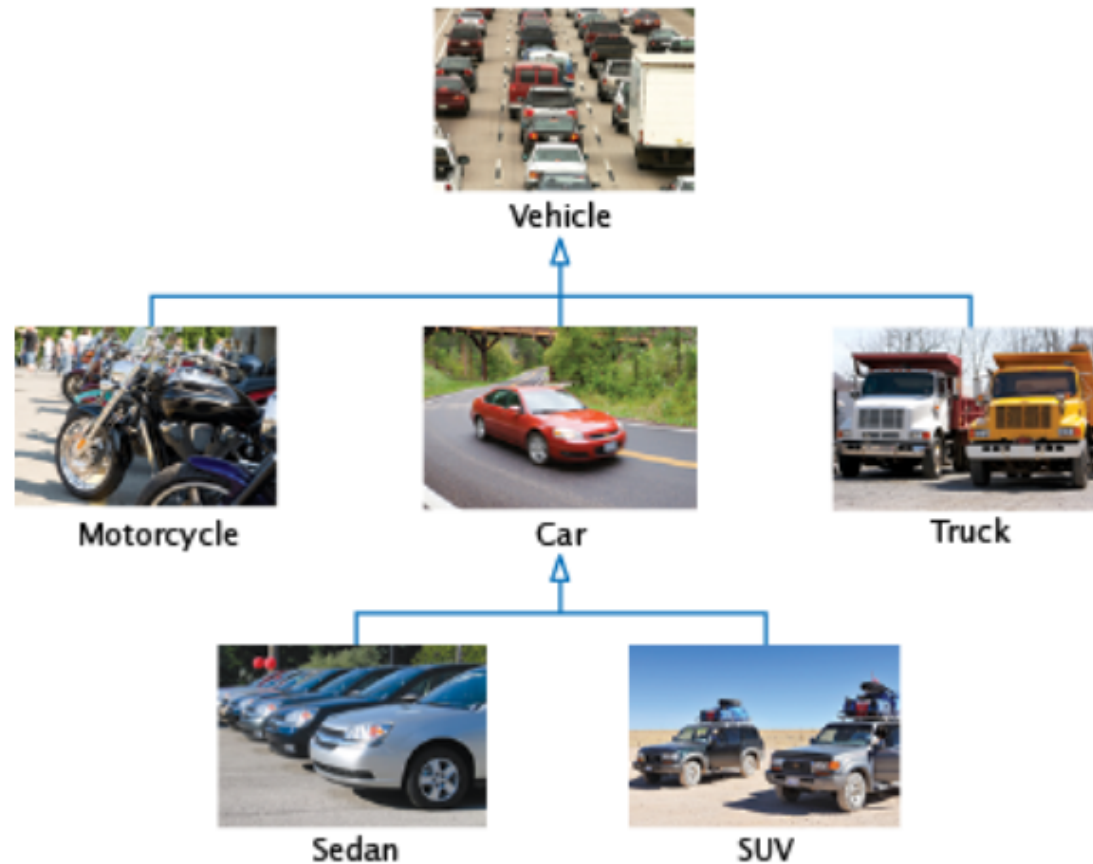
- **Code Reuse**
 - Avoid writing the same code multiple times.
 - Common attributes and methods are written once in the base class and reused in all derived classes.
- **Extensibility and Specialization**
 - Derived classes can add new features or override existing ones to create specialized behavior.
 - Example: Bird might override move() from Animal to "fly".
- **Maintainability**
 - Changes in the base class automatically propagate to derived classes.
 - Easier to update or fix code in one place.
- **Polymorphism (advanced motivation) – will learn later!**
 - Enables using a base class pointer or reference to handle derived class objects — supporting flexible and dynamic behavior in programs.



Motivation for Inheritance

- Think of it like this:
- **Base class = blueprint** for a general idea (e.g., "Vehicle").
- **Derived class = specialized version** of that idea (e.g., "Car", "Bike").
- Each derived class *is a* type of Vehicle, but with extra details.

An inheritance hierarchy



© Richard Stouffer/iStockphoto (vehicle); © Ed Hiddle/iStockphoto (motorcycle);
© Yin Yang/iStockphoto (car); © Robert Pernel/iStockphoto (truck); nicholas
beilton/iStockphoto (sedan); © Cezary Wojtkowski/AGE Fotostock America (SUV).

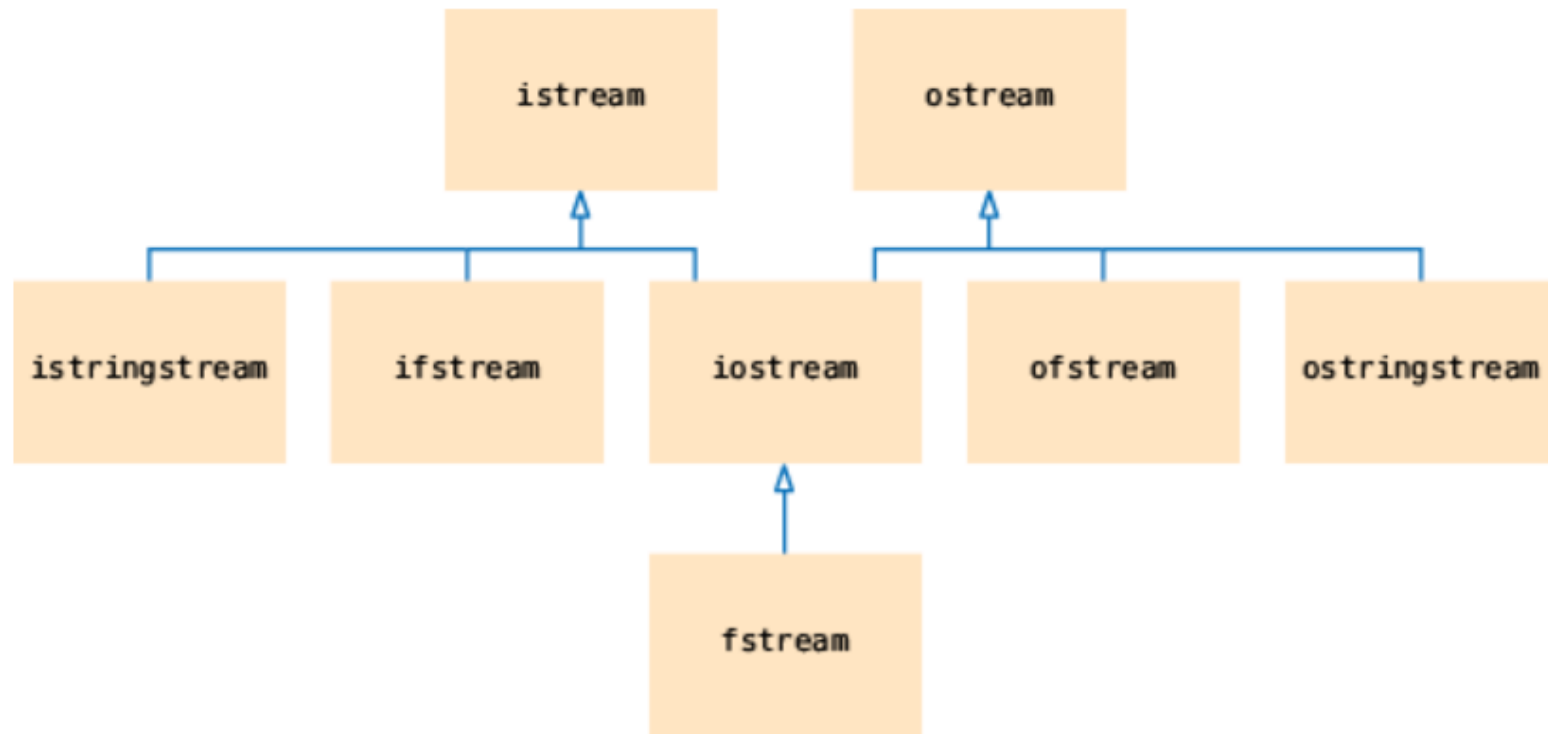
- Cars share common traits of vehicles, such as the ability to transport people.
- The class **Car** inherits from the class **Vehicle**.
- The Vehicle class is the **base class** and the Car class is the **derived class**.



Inheritance

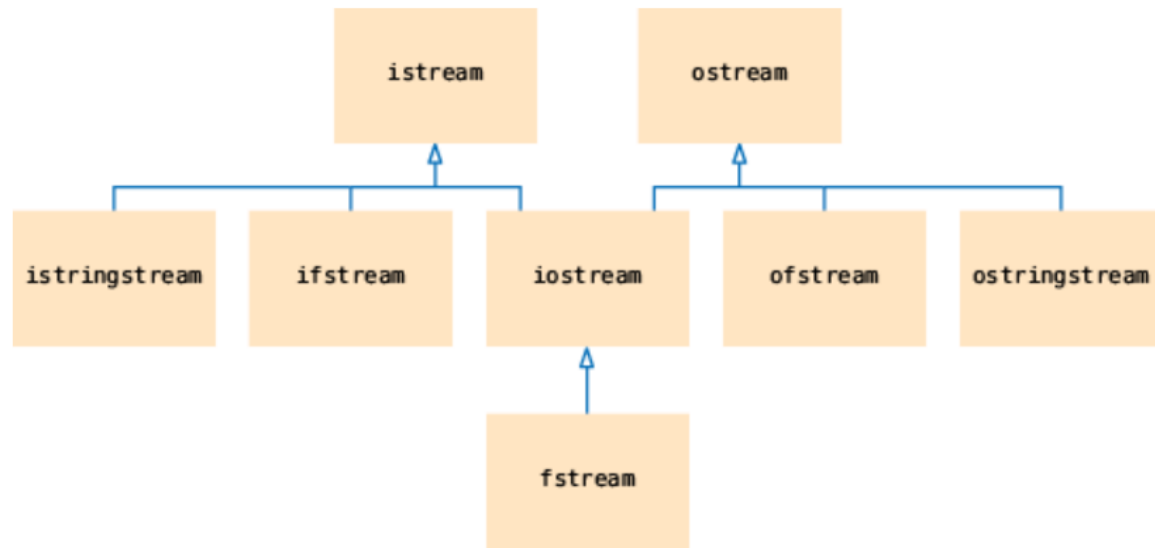
- The inheritance relationship is very powerful because it allows us to **reuse algorithms** with objects of different classes.
- Suppose we have an algorithm that manipulates a Vehicle object.
- Because a ***car*** is a special kind of ***vehicle***, we can supply a Car object to such an algorithm, and it will work correctly.
 - Here **Vehicle** is the **base class** and **Car** is a **derived class**
 - or **Vehicle** is the **parent class** and **Car** is a **child class**
- This is an example of the **Substitution principle**, which states:
- **“A derived-class object can be used anywhere a base-class object is expected.”**

Inheritance hierarchy of C++ IO Classes



- Let's look at an example!
- The diagram shows how C++ input/output (I/O) classes are organized through inheritance.
- Each **rectangle** represents a **class** in the I/O hierarchy.
- The **hollow upward arrow** indicates an **inheritance relationship** — meaning the class below **inherits** from the class above it.

Inheritance hierarchy of C++ IO Classes



- `istream` — handles input (e.g., reading from keyboard or file).
- `ostream` — handles output (e.g., writing to screen or file).
- `iostream` — inherits from both `istream` and `ostream`, allowing both input and output.
- Specialized stream classes extend these:
 - `ifstream` — input from files.
 - `ofstream` — output to files.
 - `fstream` — input/output from files.
 - `istringstream`, `ostreamstream`,
`stringstream` — input/output from strings (in memory).



Self-Test

- 1) Suppose you are to design an inheritance hierarchy with the following classes: Snake, Crocodile, Reptile, and Turtle. Which of these is the base class?
- ☐ Snake
 - ☐ Crocodile
 - ☐ Reptile
 - ☐ Turtle



Self-Test

1) Suppose you are to design an inheritance hierarchy with the following classes: Snake, Crocodile, Reptile, and Turtle. Which of these is the base class?

- ☐ Snake
- ☐ Crocodile
- ☐ Reptile
- ☐ Turtle

Answer: Reptile

Reptile is the general category (base class). Snake, Crocodile, and Turtle are specific kinds of reptiles (derived classes).



Self-Test

Suppose you have designed an inheritance hierarchy that includes the following relationships:

Guitar is derived from Instrument

AcousticGuitar is derived from Guitar

ElectricGuitar is derived from Guitar

1) Given the declarations below, which of the objects CANNOT be passed to the function `tune (Guitar& g)`?

- ☐ `AcousticGuitar ag;`
- ☐ `ElectricGuitar eg;`
- ☐ `Guitar my_guitar;`
- ☐ `Instrument my_instrument;`



Self-Test

Suppose you have designed an inheritance hierarchy that includes the following relationships:

Guitar is derived from Instrument

AcousticGuitar is derived from Guitar

ElectricGuitar is derived from Guitar

1) Given the declarations below, which of the objects CANNOT be passed to the function `tune (Guitar& g)`?

- ☐ `AcousticGuitar ag;`
- ☐ `ElectricGuitar eg;`
- ☐ `Guitar my_guitar;`
- ☐ `Instrument my_instrument;`

Answer: **Instrument my_instrument**

The parameter `g` must be a Guitar object or derived from Guitar.

You can pass Guitar, AcousticGuitar, or ElectricGuitar, because they are “is-a Guitar”.

Questions Hierarchy - Example

- Consider a simple hierarchy of questions in a quiz. A quiz consists of different kinds of questions such as:
 - Fill-in-the-blank
 - Choice (single or multiple)
 - Numeric (where an approximate answer is ok; e.g., 1.33 when the actual answer is $4/3$)
 - Free response
- An inheritance hierarchy for these question types can be constructed.



Questions Hierarchy - Example

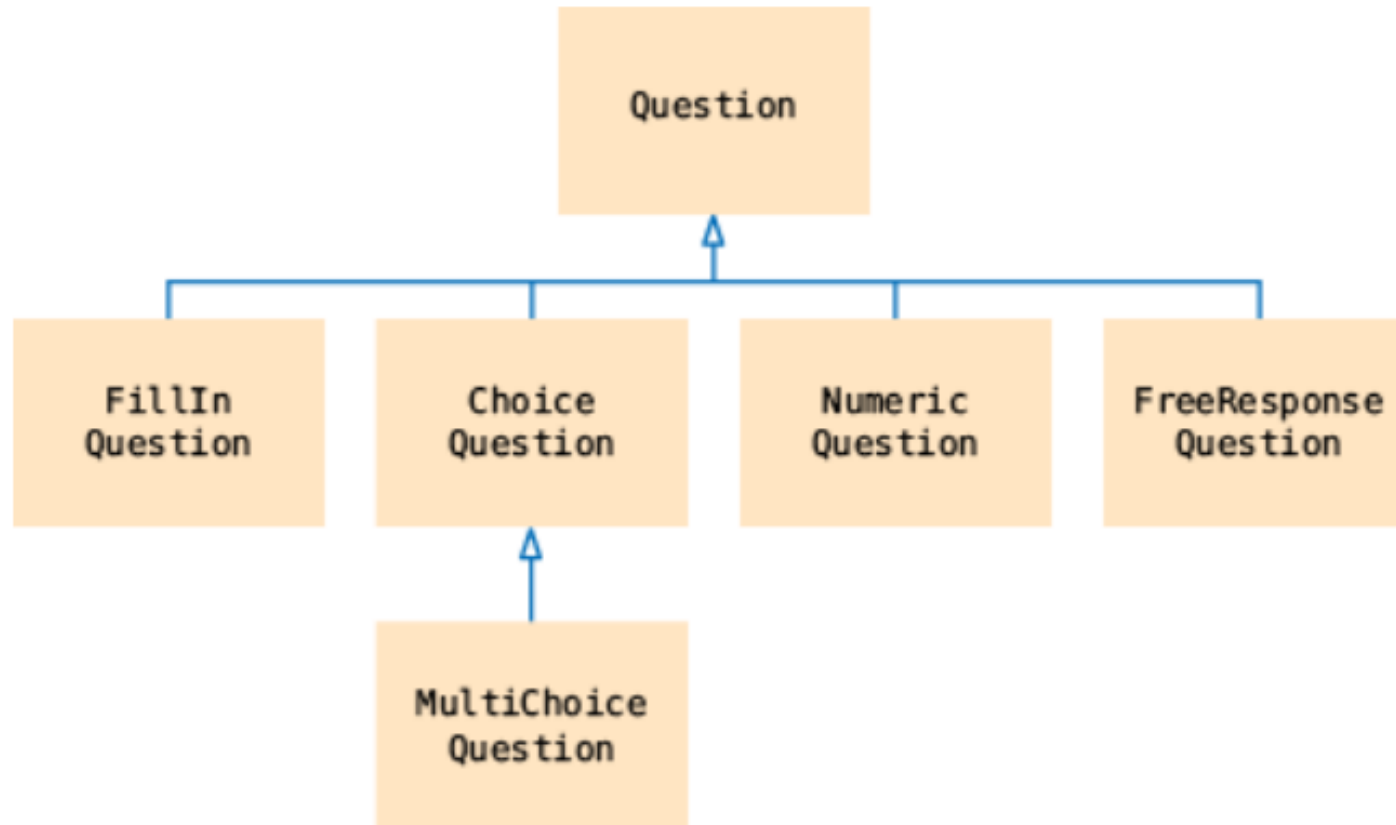
- **Object-Oriented Design Insight:**

- All these question types share **common behavior** (e.g., displaying a question, checking an answer).
- Therefore, we can define a **base class** Question containing shared data and methods.
- Each **specific question type** is implemented as a **derived class** that extends or customizes Question.

- **Inheritance Hierarchy**

- **Base class:** Question
- **Derived classes:**
 - FillInQuestion
 - ChoiceQuestion → extended by MultiChoiceQuestion
 - NumericQuestion
 - FreeResponseQuestion

Questions Hierarchy - Example



Key Takeaway: Inheritance allows us to **reuse code** for shared functionality while enabling **specialized behavior** for each question type.



The Question class

- At the root of this hierarchy is the class `Question`.
- A question can display its text, and it can check whether a given response is a correct answer.

```
class Question
{
private:
    string text;
    string answer;
public:
    Question(); // constructor
    void set_text(string question_text);
    void set_answer(string
correct_response);
    bool check_answer(string response)
const;
    void display() const;
};
```



Derived Classes

- In C++, you form a derived class from a base class by specifying what makes the derived class different.
- You define the member functions **that are new to the derived class**.
- The derived class automatically **inherits all member functions** from the base class,
- You **can change the implementation** if the inherited behavior is not appropriate.
- You only define the added data members.

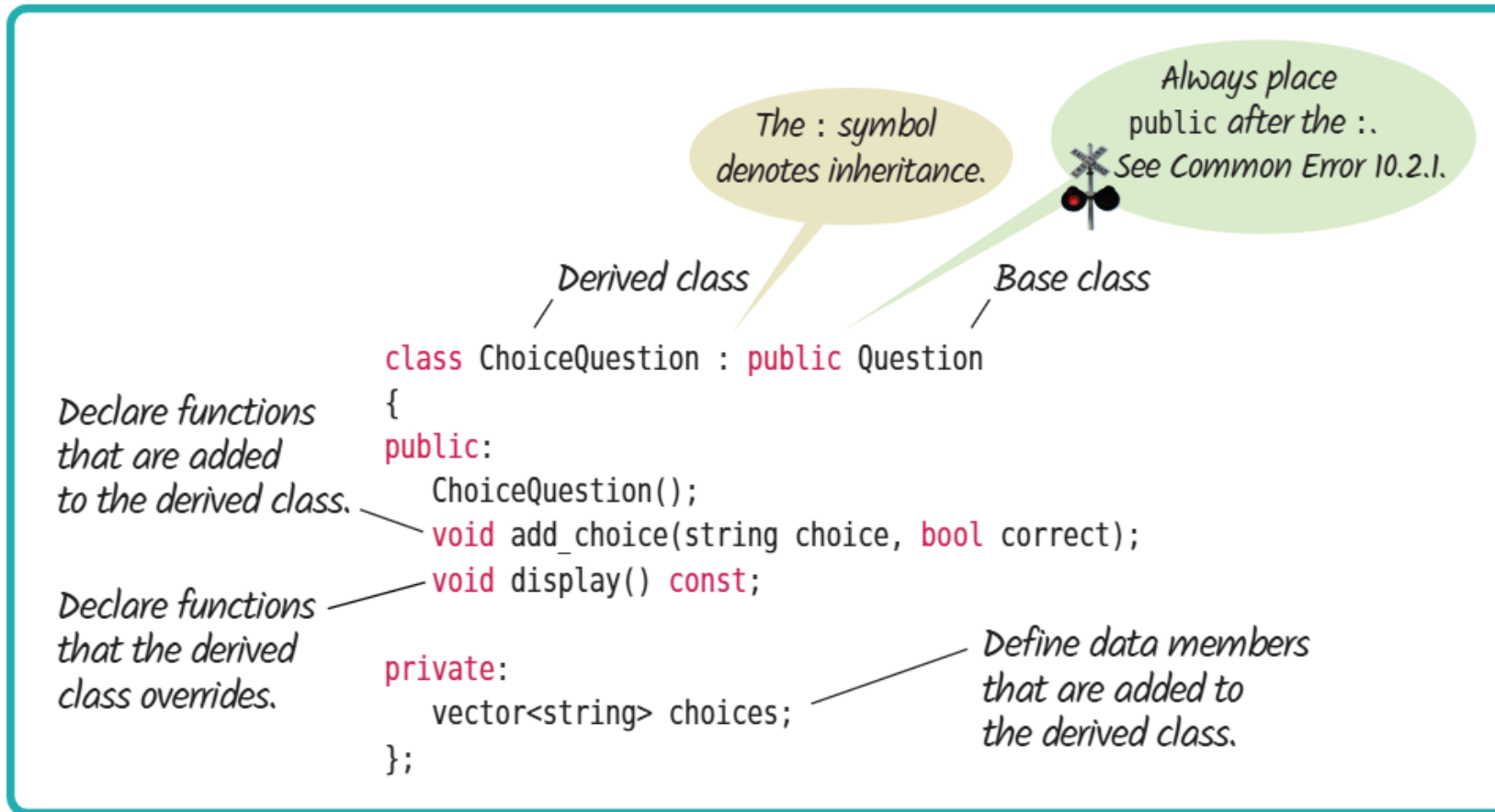


Derived Classes – ChoiceQuestion

- Here is the syntax for the definition of a derived class:

```
class ChoiceQuestion : public Question
{
public:
    New and changed member functions
private:
    Additional data members
};
```

Derived Classes – Format



Derived Class - ChoiceQuestion

- A ChoiceQuestion object differs from a Question object in three ways:
 - Its object store the various choices for the answer. **An array** in addition to just text
 - There is a member function for adding another choice.
 - The **display function** of the ChoiceQuestion class shows these choices so that the respondent can choose one of them.
 - Note that display() was also part of Question class, here we would override it's meaning!

```
class ChoiceQuestion : public
Question
{
private:
    vector<string> choices;
public:
    ChoiceQuestion();
    void add_choice(string choice,
                    bool correct);
    void display() const;
};
```





Derived Class - ChoiceQuestion

- The `add_choice` function is specific to the `ChoiceQuestion` class.
- You can only apply it to `ChoiceQuestion` objects, not general `Question` objects.
- However, the `display` function is a redefinition of a function that exists in the base class; it is redefined to take into account the special needs of the derived class.
- We say that the derived class **overrides** this function.
- (Definition of **override**: Redefine a function from a base class in a derived class.)



Derived Class - ChoiceQuestion

- The `add_choice` function is a simple function to append choices to the choice vector

```
void ChoiceQuestion::add_choice(string choice, bool
correct)
{
    choices.push_back(choice); //choices is a vector
}
```

Remember `::` is a **scope resolution operator** i.e. function is defined outside the class



Derived Class - ChoiceQuestion

- Overridden display() function

```
void ChoiceQuestion::display() const {
    Question::display(); // Call base class display to show the question
    text
    cout << "Choices>> ";           // Label before listing choices

    for (int i = 0; i < choices.size(); i++) {
        cout << choices[i] << " ";   // Print each choice using index
    }
    cout << endl;                    // New line at the end
}
```



Derived Class - NumericQuestion

- A **NumericQuestion** object differs from a **Question** object in the following ways:
 - It stores a **numeric answer** instead of text i.e. see **double expected**;
 - It overrides the `check_answer()`
 - Adds a new function `set_answer()`
 - It overrides the `display()` function.
 - Note that `display()` was also part of `Question` class, here we would override it's meaning!

```
class NumericQuestion : public Question
{
public:
    void display() const;
    bool check_answer(string response) const;
    void set_answer(double correct_response);
private:
    double expected;
};

void NumericQuestion::display() const
{
    Question::display();
    cout << "Enter a number." << endl;
}
```



Derived Class - NumericQuestion

- **Overrides**
check_answer() →
converts input to double
and checks closeness
- **Adds set_answer()** to
store the expected
numeric response

```
bool NumericQuestion::check_answer(string  
response) const  
{  
    double value = stod(response);  
    //return true if value and  
    //expected are very close  
    return abs(value - expected) < 1E-12;  
}  
  
void NumericQuestion::set_answer(double  
correct_response)  
{  
    answer = correct_response;  
}
```



Logistics – Midterm grades

- Midterm has been graded. Feel free to discuss the grades during this week's office hours.
- Some statistics:
 - MCQs mean: 10.6 out of 15
 - MCQs median 10
 - Coding mean: 28.7 out of 35
 - Coding median: 33.25
 - Total mean: 39.21 out of 50
 - Total median: 43.25 out of 50
- You can calculate the final score by adding your MCQs and coding component. Final score out of 20 is $\text{sum} \times 20 / 50$
- We will update that on canvas once all sections have entered their grades.



Logistics – Class Quiz and Project

- Class quiz 3 this Wednesday
- **Content for the quiz:**
 - Classes and Objects
 - Operator Overloading
 - Linkedlist
- **Format of the quiz:**
 - Five questions, 20 minutes, we will start at 10:05 am sharp.
 - No extra time will be given.
 - Come prepared this Wednesday!
- **Note:** We will take one extra class and lab quiz and do best of 3 for each class and lab quizzes



Summary last week!

- Inheritance allows us to **reuse algorithms** with objects of different classes.
- The derived class **inherits** both **data** and **behavior (functions)** from the base class.
- We learnt the syntax `class ChoiceQuestion : public Question`
 - Where `ChoiceQuestion` is derived from `Question`
 - Note the `:` symbol and `public` keyword
- We can define new data members as well as override existing functions that we have inherited in the derived class.
- Example: Base Class `Question` with derived classes `Choice Question` and `Numeric Question`
- Takeaway home exercise for you: Try to use these two derived classes + one parent class and write a main function to test various inputs!

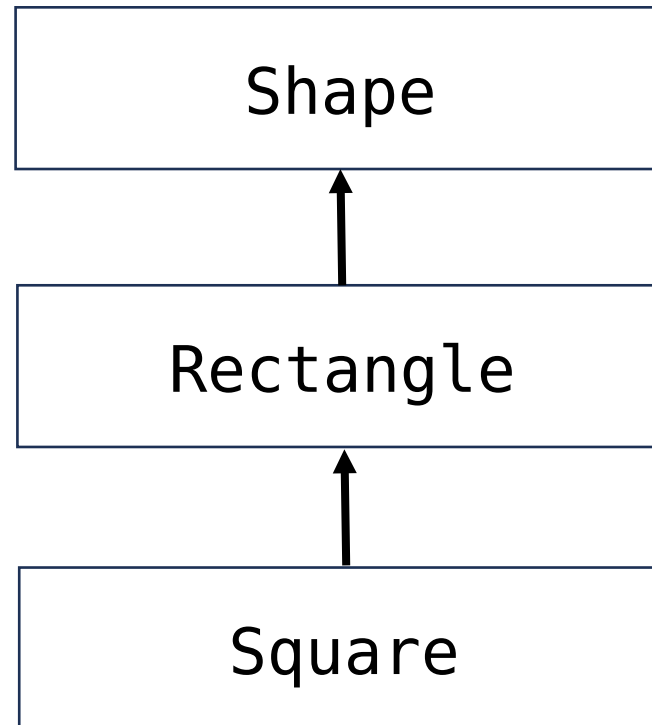


Let's move to VS Code!

- Let's move to VS Code to see the main of Question Class!

Another Example

- Consider the following inheritance hierarchy:





The Shape Example – Object Creation Order

- Let's create a **Shape Class Hierarchy**.
- On the top most level, we have a **class Shape**
- Class Rectangle is **derived** from Shape.
- Next we will create class Square **derived** from class Rectangle.

```
class Shape {  
private:  
    string name;  
public:  
    Shape() {  
        cout << "Creating shape" << endl;  
    }  
};  
class Rectangle : public Shape {  
private:  
    int length;  
    int width;  
public:  
    Rectangle() {  
        cout << "Creating rectangle" << endl;  
    }  
};
```

The Shape Example – Object Creation Order

- When a derived object is created, **base class constructors run first**.
- The order of construction is: **Base class → Derived class → Constructor body**.
- In this example:
 - Shape() runs first → prints *"Creating shape"*.
 - Rectangle() runs next → prints *"Creating rectangle"*.
- Destruction happens in the **reverse order** (Derived → Base)

```
class Square : public Rectangle {  
public:  
    Square() {  
        cout << "Creating square" <<  
endl;  
    }  
};  
  
int main() {  
    Square square;  
    return 0;  
}
```

Didn't we initialize Square only in main? So why are we seeing this output?

```
Creating shape  
Creating rectangle  
Creating square
```



Object Creation Order – General Rule

- **Key Idea:**
 - Object creation always starts from the **base class upward**, ensuring all inherited parts are properly initialized
- When creating an object of a **derived class**, the **base class constructor** is called **first**.
- This process continues **up the inheritance chain** until the **topmost base class** is constructed.
- Example: Creating a Square object → calls Rectangle() → calls Shape().
- Q. What happens in a constructor of a base class is not present?



Object Creation Order – General Rule

- **Key Idea:**
 - Object creation always starts from the **base class upward**, ensuring all inherited parts are properly initialized
- When creating an object of a **derived class**, the **base class constructor** is called **first**.
- This process continues **up the inheritance chain** until the **topmost base class** is constructed.
- Example: Creating a Square object → calls Rectangle() → calls Shape().
- Q. What happens in a constructor of a base class is not present?
 - If a **base class constructor** is **not explicitly called**, the **default (no-argument) constructor** is invoked automatically.



Constructors in Inheritance

```
class Shape {  
private:  
    string name;  
  
public:  
    Shape(string name) {  
        this->name = name;  
        cout << "Creating shape" <<  
endl;  
    }  
};
```

```
class Rectangle : public Shape {  
private:  
    int length;  
    int width;  
  
public:  
    Rectangle(string name, int length, int  
width) {  
        this->name = name; //Can I do this?  
        this->length = length;  
        this->width = width;  
        cout << "Creating rectangle" <<  
endl;  
    }  
};
```

Question: We want to create a rectangle; and we want to assign it a name, i.e. initialize its name attribute from its constructor. Can we do this directly as shown here?



Constructors in Inheritance

- The correct way to initialize a parent(super/base) class's variables is to call its constructor, since the variables of the parent(super/base) class are private.
- This can be done using an initializer list as follows:

```
class Shape {  
private:  
    string name;  
  
public:  
    Shape(string n) : name(n) {  
        cout << "Now creating shape with name: " << name <<  
endl;  
    }  
};
```

Constructors in Inheritance - Continued

```
class Rectangle : public Shape {  
private:
```

```
    int length;  
    int width;
```

```
public:
```

```
    Rectangle(string n, int l, int w) : Shape(n), length(l),  
width(w) {  
        cout << "Now creating rectangle with length " << length <<  
" and width " << width << endl;  
    }  
};
```



What's
this?



Constructors in Inheritance - Continued

```
class Square : public Rectangle {
public:
    Square(string n, int side) : Rectangle(n, side, side) {
        cout << "Now creating square with side " << side << endl;
    }
};

int main() {
    Square square("MySquare", 5);
    return 0;
}
```

Now creating shape with name: MySquare
Now creating rectangle with length 5 and width 5
Now creating square with side 5



Constructors in Base and Derived Classes

– Another Example

- **Base class** constructors initialize base members.
- **Derived class** use **initializer lists** constructors initialize both:
 - Base class members (by calling the base constructor)
 - Their own members
- The **base constructor is always called first.**

```
class Base {  
private:  
    string text;  
    string answer;  
  
public:  
    Base(string t, string a) : text(t), answer(a) {}  
};  
  
class Derived : public Base {  
  
public:  
    Derived(string t, string a)  
        : Base(t, a) {} // Calls Base constructor  
};
```



Constructors in Base and Derived Classes

Another Example

Base Class Constructor

```
class Question {  
protected:  
    string text, answer;  
public:  
    Question(string q, string a) : text(q), answer(a) {}  
};
```

Derived Class Constructor

```
class ChoiceQuestion : public Question {  
private:  
    vector<string> choices;  
public:  
    // Initialize both base and derived members  
    ChoiceQuestion(string q, string a, vector<string> ch)  
        : Question(q, a), choices(ch) {}  
};
```

In main

```
vector<string> ch = {"Python", "C++",  
                    "HTML", "JavaScript"};  
  
ChoiceQuestion cq("Which language is used for programming?",  
                  "C++", ch);
```

Key Takeaway:

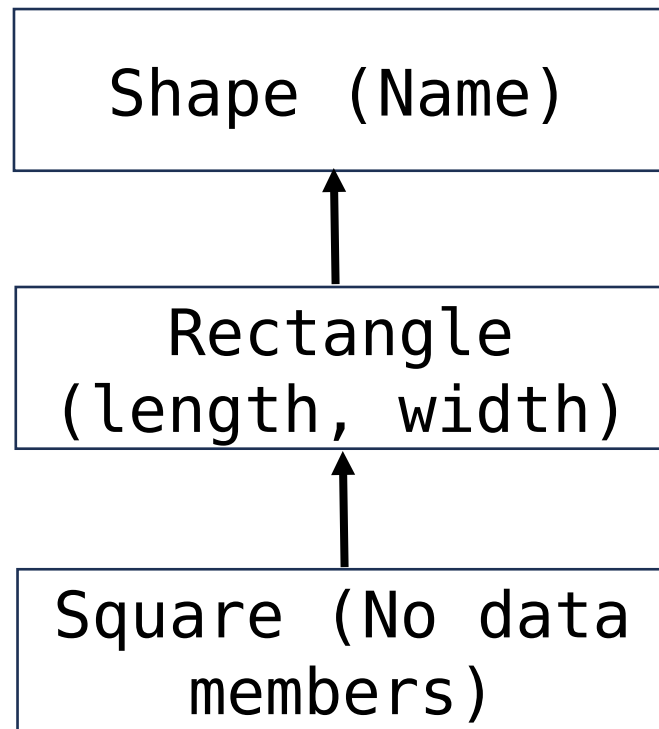
- The base class must be **initialized** before the derived class body runs.
- Use an **initializer list** to call the base constructor and initialize members.
- The derived constructor can pass values directly to the base constructor.
- Order of construction: **Base → Derived → Constructor body.**



Object Creation Order – rules

- In C++, you cannot directly call the constructor of the superclass (base class) from within the body of the constructor of the subclass (derived class).
- Instead, you must call the base class constructor in the initializer list of the derived class constructor.
- There is a specific order to follow when initializing members in the initializer list of a constructor in C++.
- The base class constructors are called before the derived class's member initializers are executed.
- If the derived class has multiple base classes, the order of initialization will follow the order in which the base classes are declared in the class definition (from left to right).

Object Creation Order – Rules



- In C++, when an object of a derived class is created, the constructors of its base classes are invoked first.
- This ensures that the base class part of the derived class object is fully initialized before any additional initialization is executed.
- The memory layout of a derived class object includes the memory for its base class parts as well.



Object Destruction Order - Reverse

- In C++, the order of object destruction follows the **reverse** of the construction order.
- When an object is destroyed, the destructors are called in the following order:
- The destructor of the derived class is called first. This allows the derived class to perform any cleanup that is specific to itself.
- After the derived class destructor has completed its execution, the destructor of the base class is called.
- This ensures that any resources or states managed by the base class are properly cleaned up.



Order of Destructors – Program

```
class Base {
public:
    Base() { std::cout << "Base constructor called." << std::endl; }
    ~Base() { std::cout << "Base destructor called." << std::endl; }
};

class Derived : public Base {
public:
    Derived() { std::cout << "Derived constructor called." << std::endl; }
    ~Derived() { std::cout << "Derived destructor called." << std::endl; }
};

int main() {
    Derived d; // Creating an object of Derived class
    return 0;
}
```

Base constructor called.
Derived constructor called.
Derived destructor called.
Base destructor called.



Class Exercise

- We saw the Shape-Rectangle-Square hierarchy.
- Add a function called `int getArea()` to this hierarchy? Which class should it be in?
- Should the square class override its `getArea()` over the Rectangle's `getArea()`?
- Should the Shape class have a `getArea()` function? If so, what should it return?
- -Review Exercises from Horstmann, Chapter 10.



Summary!

- Object order creation always happens from Base Class to Derived Class
- Since we cannot access private data members of Base Class in a Derived Class constructor, we instead use initializer list in a constructor of Derived Class
- Order of Destructors happens in reverse i.e. Derived Class Object Destructors are called first then Base Class Destructors.